Lab: From a new URDF to a trained policy (AgileX PiPER picks up a cube)

In Train SO101 to pick up a cube you followed a finished example from the SO101 URDF to a trained policy. In this lab you do the whole process yourself for a robot that the project does not know yet: the **AgileX PiPER**, a 6-joint desktop arm with a two-finger gripper.

You start from the manufacturer's URDF on GitHub and finish with a policy that reaches a cube, grasps it, lifts it and carries it to a target, played back in the Isaac Sim GUI.

```
A. Robot                B. Task                C. Train             D. Play
fix URDF -> inspect  -->  scene, actions,   -->  PPO, 1024 arms  -->  trained policy
robot config -> test      rewards, register      in parallel          in the Isaac Sim GUI
(Steps 1-5)               (Steps 6-9)            (Steps 10-12)        (Steps 13-14)
```

Learning outcomes

By the end of this lab you can:

1. Prepare a robot description written for ROS so that Isaac Lab can load it.
2. Read a URDF and find the names a task needs: joints, root link, end-effector link.
3. Write an Isaac Lab robot config (`ArticulationCfg`) with sensible start pose and motor drives, and test it before any training.
4. Adapt an existing manipulation task to a new robot: actions, end-effector frame, object, reach ranges and reward thresholds.
5. Register, train, monitor and play back a reinforcement-learning policy.

Why the PiPER

	SO101 (topic 3)	AgileX PiPER (this lab)
Source of the URDF	Ships inside `isaac_so_arm101`, ready to use	ROS package on GitHub (agilexrobotics/piper_ros, MIT licence)
Mesh paths	Relative (`assets/...stl`)	ROS `package://` URLs: must be rewritten
Root link	`base_link`	Empty `dummy_link` above `base_link`: must be removed
Arm joints	5 revolute, named after their function	6 revolute, `joint1` … `joint6`
Gripper	1 revolute jaw (0 to 0.5 rad)	2 prismatic fingers: `joint7` 0 to 0.035 m, `joint8` -0.035 to 0 m
Mass / reach	about 0.8 kg / 0.35 m	about 4.7 kg / 0.6 m

Every row in this table is something you must handle yourself. That is the point of the lab: it shows which parts of a task depend on the robot.

Time

Part	Time
Steps 1–9 (robot and task)	about 2 hours in the lab
Step 11 (full training)	1 hour or more, unattended (run it after the lab if needed)
Steps 13–14 (play back)	15 minutes

What you need

Item	Value
Software	Isaac Sim 5.1, Isaac Lab 2.3, `isaac_so_arm101` installed. If any is missing, follow Appendix A
Python to use	`C:\isaacsimlab\env_isaaclab\Scripts\python.exe`
Project folder (run commands from here)	`C:\isaacsimlab\isaac_so_arm101`
Lab files	`C:\SO101wClaudeIsaac\labs\piper-lift\` (tools and finished solution)
Internet	Step 1 downloads the PiPER URDF from GitHub; the first run downloads the table and cube from NVIDIA
Before you start	Complete topic 3 (Train SO101 to pick up a cube), at least Steps 1–9

Lab files provided:

```
C:\SO101wClaudeIsaac\labs\piper-lift\
    tools\prepare_piper_urdf.py     Step 1: download and fix the URDF
    tools\inspect_urdf.py           Step 2: print links and joints as tables
    tools\test_piper_joints.py      Step 5: move every joint in Isaac Sim
    tools\list_lab_envs.py          Step 8: check the task is registered
    tools\install_lab_files.ps1     instructor: copy the finished solution in one go
    package\robots\agilex_piper\    solution: robot config (Step 4)
    package\tasks\lift_piper\       solution: task, training config, registration (Steps 6-8)
```

Set up every terminal

```
cd C:\isaacsimlab\isaac_so_arm101
$py  = "C:\isaacsimlab\env_isaaclab\Scripts\python.exe"
$lab = "C:\SO101wClaudeIsaac\labs\piper-lift"
$pkg = "C:\isaacsimlab\isaac_so_arm101\src\isaac_so_arm101"
```

Keep the `&` in front of `$py` in every command (`& $py …`).

Before you add files to `isaac_so_arm101`, run `git status` there and make sure you know what is already changed. Your new files go in two new folders, so they do not touch the SO101 tasks.

Stage A: The robot

Step 1: Get the PiPER URDF and fix it for Isaac Lab

The PiPER description was written for ROS. Three things must change before Isaac Lab can load it:

Problem in the original	Why it matters	What the script does
Mesh paths like `package://piper_description/meshes/link1.STL`	`package://` is a ROS address. Isaac Lab cannot find the files	Rewrites them to `meshes/link1.stl` (relative to the URDF)
Mesh files end in `.STL`	Upper-case extensions are not reliably recognised	Saves the meshes as `.stl`
Root link `dummy_link`, fixed to `base_link`	Isaac Lab merges links joined by fixed joints (`merge_fixed_joints=True` by default), so `base_link` would disappear into	Removes `dummy_link` and its joint

Problem in the original	Why it matters	What the script does

`dummy_link`. The task refers to `base_link`

Create the robot folder and run the script:

```
New-Item -ItemType Directory -Force "$pkg\robots\agilex_piper\urdf"
& $py "$lab\tools\prepare_piper_urdf.py" --dest "$pkg\robots\agilex_piper\urdf"
```

You should see:

```
[1/4] Getting the original URDF and meshes
[2/4] Rewriting mesh paths
      20 mesh references rewritten
[3/4] Removing dummy_link
[4/4] Saved ...\robots\agilex_piper\urdf\piper.urdf
      root link: base_link  |  links: 10  |  joints: 9
```

No internet on the lab PC? Your instructor can give you a copy of `piper_ros\src\piper_description`. Add `--source <that folder>` to the command.

The script (for reference; you do not need to type it)

```
"""Download the AgileX PiPER URDF and make it ready for Isaac Lab.

The PiPER description comes from the ROS package `piper_description` in
https://github.com/agilexrobotics/piper_ros (MIT licence). It was written for
ROS, so three things must change before Isaac Lab can load it:

  1. Mesh paths use ROS `package://piper_description/meshes/...` URLs.
     Isaac Lab does not know ROS packages, so we rewrite them to plain
     relative paths: `meshes/link1.stl`.
  2. Mesh files end in `.STL` (upper case). We save them as `.stl`.
  3. The root link is an empty `dummy_link` joined to `base_link` by a fixed
     joint. We remove it so `base_link` is the root, which is what the
     task config refers to.

Usage (no Isaac Sim needed; any Python 3.9+ works, including env_isaaclab):

    & $py prepare_piper_urdf.py --dest <folder>

    --dest     Output folder. Gets piper.urdf and a meshes\\ subfolder.
    --source   Optional local copy of piper_ros\\src\\piper_description
               (use this if the lab PCs have no internet access).
"""

import argparse
import shutil
import sys
import urllib.request
import xml.etree.ElementTree as ET
from pathlib import Path

REPO_RAW = "https://raw.githubusercontent.com/agilexrobotics/piper_ros/master/src/piper_description"
URDF_NAME = "piper_description.urdf"
MESHES = [
    "base_link", "link1", "link2", "link3", "link4", "link5", "link6",
    "gripper_base", "link7", "link8",
]



def fetch(rel_path: str, out: Path, source: Path | None) -> None:
    """Copy a file from the local source folder, or download it from GitHub."""
    if source is not None:
        shutil.copyfile(source / rel_path, out)
        return
    url = f"{REPO_RAW}/{rel_path}"
```

```
    print(f"  downloading {url}")
    with urllib.request.urlopen(url, timeout=60) as resp, open(out, "wb") as f:
        f.write(resp.read())



def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("--dest", required=True, help="Output folder for piper.urdf and meshes/")
    parser.add_argument("--source", default=None, help="Local piper_description folder (optional)")
    args = parser.parse_args()

    dest = Path(args.dest)
    source = Path(args.source) if args.source else None
    mesh_dir = dest / "meshes"
    mesh_dir.mkdir(parents=True, exist_ok=True)

    # 1. Get the original URDF and the 10 meshes it uses
    original = dest / "piper_description_original.urdf"
    print("[1/4] Getting the original URDF and meshes")
    fetch(f"urdf/{URDF_NAME}", original, source)
    for name in MESHES:
        fetch(f"meshes/{name}.STL", mesh_dir / f"{name}.stl", source)

    # 2. Rewrite mesh paths: package://piper_description/meshes/link1.STL -> meshes/link1.stl
    print("[2/4] Rewriting mesh paths")
    tree = ET.parse(original)
    robot = tree.getroot()
    changed = 0
    for mesh in robot.iter("mesh"):
        filename = mesh.get("filename", "")
        stem = Path(filename.replace("package://", "")).stem
        mesh.set("filename", f"meshes/{stem}.stl")
        changed += 1
    print(f"      {changed} mesh references rewritten")

    # 3. Remove the empty root link so base_link becomes the root
    print("[3/4] Removing dummy_link")
    for joint in list(robot.findall("joint")):
        if joint.find("parent").get("link") == "dummy_link":
            robot.remove(joint)
    for link in list(robot.findall("link")):
        if link.get("name") == "dummy_link":
            robot.remove(link)

    # 4. Save and check that every mesh file exists
    out_urdf = dest / "piper.urdf"
    tree.write(out_urdf, encoding="utf-8", xml_declaration=True)
    print(f"[4/4] Saved {out_urdf}")
    missing = [m.get("filename") for m in robot.iter("mesh") if not (dest / m.get("filename")).is_file()]
    if missing:
        sys.exit(f"ERROR: missing mesh files: {missing}")
    roots = {l.get("name") for l in robot.findall("link")} - {j.find("child").get("link") for j in robot.findall("joint")}
    print(f"      root link: {', '.join(sorted(roots))}  |  links: {len(robot.findall('link'))}  |  joints:
{len(robot.findall('joint'))}")
    print("Done. Next: inspect it with inspect_urdf.py")



if __name__ == "__main__":
    main()
```

Step 2: Inspect the URDF and find the names

```
& $py "$lab\tools\inspect_urdf.py" "$pkg\robots\agilex_piper\urdf\piper.urdf"
```

Expected output:

```
Robot: piper   file: C:\isaacsimlab\isaac_so_arm101\src\isaac_so_arm101\robots\agilex_piper\urdf\piper.urdf


LINKS (10)
  name             mass (kg)  mesh
  base_link            1.020  meshes/base_link.stl
```

```
  link1               0.710  meshes/link1.stl
  link2               1.170  meshes/link2.stl
  link3               0.500  meshes/link3.stl
  link4               0.380  meshes/link4.stl
  link5               0.383  meshes/link5.stl
  link6               0.007  meshes/link6.stl
  gripper_base        0.450  meshes/gripper_base.stl
  link7               0.025  meshes/link7.stl
  link8               0.025  meshes/link8.stl
  TOTAL               4.670

JOINTS (9)
  name                  type       parent -> child                lower   upper  unit
  joint1                revolute   base_link -> link1            -2.618   2.618  rad
  joint2                revolute   link1 -> link2                 0.000   3.140  rad
  joint3                revolute   link2 -> link3                -2.967   0.000  rad
  joint4                revolute   link3 -> link4                -1.745   1.745  rad
  joint5                revolute   link4 -> link5                -1.220   1.220  rad
  joint6                revolute   link5 -> link6                -2.094   2.094  rad
  joint6_to_gripper_base fixed     link6 -> gripper_base              -       -
  joint7                prismatic  gripper_base -> link7          0.000   0.035  m
  joint8                prismatic  gripper_base -> link8         -0.035   0.000  m

Root link: ['base_link']
Movable joints (8): ['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6', 'joint7', 'joint8']
```

Task A1. Use the output to fill in this table. You will type these names into the configs in Steps 4 and 6.

Needed for	Name(s)
Arm joints (arm action, arm actuators)	
Finger joints (gripper action, gripper actuator)	
Open position of each finger	
Root link (end-effector frame sensor starts here)	
Wrist link to measure the grasp point from	

Question A2. The URDF lists 9 joints but only 8 can move. Which one is fixed, and what happens to it in Isaac Lab?

Step 3 (optional): Convert to USD and look at the robot

Isaac Lab converts the URDF automatically when a task starts, so this step is not needed for training. Do it to see the robot and check the meshes.

```
& $py C:\isaacsimlab\IsaacLab\scripts\tools\convert_urdf.py `
    "$pkg\robots\agilex_piper\urdf\piper.urdf" `
    C:\isaacsimlab\piper_usd\piper.usd `
    --fix-base --joint-stiffness 100 --joint-damping 1
```

- Isaac Sim opens and shows the PiPER. Press **Play**: it should stand still.
- In the **Stage** panel, find `joint2` and change its drive target in the **Property** panel to see the shoulder move.
- Close the window when you are done. Add `--headless` to convert without the GUI.

Step 4: Write the robot config (`PIPER_CFG`)

Create two files in `$pkg\robots\agilex_piper\`.

`__init__.py`:

```
"""AgileX PiPER robot description and Isaac Lab config."""

from .piper import PIPER_CFG  # noqa: F401
```

piper.py:

```
"""Isaac Lab robot config for the AgileX PiPER arm (6 arm joints + 2-finger gripper).

The URDF in urdf/piper.urdf is made by tools/prepare_piper_urdf.py from the
official piper_description package (https://github.com/agilexrobotics/piper_ros,
MIT licence). Isaac Lab converts it to USD automatically when a task starts.

Names used below come from the URDF (check them with tools/inspect_urdf.py):
    arm joints     joint1 ... joint6        (revolute, rad)
    finger joints  joint7 (0 .. 0.035 m), joint8 (-0.035 .. 0 m)   (prismatic)
    root link      base_link
    wrist link     link6  (gripper_base is fixed to it)
"""

from pathlib import Path

import isaaclab.sim as sim_utils
from isaaclab.actuators import ImplicitActuatorCfg
from isaaclab.assets.articulation import ArticulationCfg

PIPER_ASSETS_DIR = Path(__file__).resolve().parent

PIPER_CFG = ArticulationCfg(
    spawn=sim_utils.UrdfFileCfg(
        asset_path=f"{PIPER_ASSETS_DIR}/urdf/piper.urdf",
        fix_base=True,  # bolt base_link to the table
        activate_contact_sensors=False,
        rigid_props=sim_utils.RigidBodyPropertiesCfg(
            disable_gravity=False,
            max_depenetration_velocity=5.0,
        ),
        articulation_props=sim_utils.ArticulationRootPropertiesCfg(
            enabled_self_collisions=False,  # the CAD fingers touch the gripper base; keep it simple
            solver_position_iteration_count=8,
            solver_velocity_iteration_count=0,
        ),
        # Drive gains are set by the actuators below, so the converter writes zero gains.
        joint_drive=sim_utils.UrdfConverterCfg.JointDriveCfg(
            gains=sim_utils.UrdfConverterCfg.JointDriveCfg.PDGainsCfg(stiffness=0, damping=0)
        ),
    ),
    init_state=ArticulationCfg.InitialStateCfg(
        pos=(0.0, 0.0, 0.0),
        rot=(1.0, 0.0, 0.0, 0.0),
        # Start pose: gripper pointing down, about 15 cm above the cube, fingers open.
        # (Forward kinematics of the URDF puts the grasp point at x=0.30 m, z=0.175 m.)
        joint_pos={
            "joint1": 0.0,
            "joint2": 1.4,
            "joint3": -1.1,
            "joint4": 0.0,
            "joint5": 1.05,
            "joint6": 0.0,
            "joint7": 0.035,
            "joint8": -0.035,
        },
        joint_vel={".*": 0.0},
    ),
    actuators={
        # Shoulder and elbow carry the most mass (about 3 kg beyond joint 2): stiffer drives.
        "shoulder_elbow": ImplicitActuatorCfg(
            joint_names_expr=["joint[1-3]"],
            effort_limit_sim=50.0,
            velocity_limit_sim=3.0,
            stiffness=400.0,
            damping=40.0,
        ),
        # The wrist only moves the gripper (about 0.5 kg).
```

```
        "wrist": ImplicitActuatorCfg(
            joint_names_expr=["joint[4-6]"],
            effort_limit_sim=15.0,
            velocity_limit_sim=3.0,
            stiffness=150.0,
            damping=15.0,
        ),
        # Prismatic fingers: stiff position drives so the grip holds the cube.
        "gripper": ImplicitActuatorCfg(
            joint_names_expr=["joint[7-8]"],
            effort_limit_sim=100.0,
            velocity_limit_sim=0.2,
            stiffness=2000.0,
            damping=100.0,
        ),
    },
    # Use the full URDF range. (0.9 would stop the fingers at 0.0315 m, short of "open" = 0.035 m.)
    soft_joint_pos_limit_factor=1.0,
)
"""AgileX PiPER with position-controlled joints, base fixed at the origin."""
```

How the values were chosen:

Setting	Value	Reason
`fix_base=True`		The real arm is bolted down. Without it the robot falls over
`enabled_self_collisions=False`		The CAD fingers overlap the gripper base slightly; self-collision would make them fight
`init_state.joint_pos`	joint2 1.4, joint3 -1.1, joint5 1.05, fingers open	Gripper points down about 15 cm above the cube (worked out with forward kinematics of the URDF)
`shoulder_elbow` actuator	stiffness 400, damping 40, effort 50	Joints 1–3 hold about 3 kg against gravity. With stiffness 400 the sag is about 0.02 rad
`wrist` actuator	stiffness 150, damping 15, effort 15	Joints 4–6 only move the gripper
`gripper` actuator	stiffness 2000, damping 100, effort 100, 0.2 m/s	Prismatic fingers need stiff drives to squeeze the cube (same idea as the Franka and OpenArm configs)
`soft_joint_pos_limit_factor=1.0`		With 0.9, the finger limit would stop at 0.0315 m and could never reach "open" = 0.035 m

These are simulation values, not measured from a real PiPER. They are a starting point; Exercise 3 asks you to try others.

Step 5: Test the robot before any training

Run this from the project folder. It builds a small scene with one PiPER and moves every joint through 70 % of its range, one after another:

```
& $py "$lab\tools\test_piper_joints.py"
```

- The first start can take several minutes (URDF conversion and shaders).
- Watch the arm: each joint swings in turn, then the fingers open and close.
- The terminal then prints the names and a tracking table, for example:

```
[test_piper_joints] joints: ['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6', 'joint7', 'joint8']
[test_piper_joints] body 'base_link': OK
[test_piper_joints] body 'link6': OK
```

```
  joint     lower     upper   max tracking error
  joint1   -2.618     2.618      ...
```

Checkpoint. Do not continue until both bodies show `OK` and no joint is flagged `<-- check stiffness / limits`. A flagged joint usually means the stiffness is too low for the mass it carries, or the joint hits something.

The script (for reference)

```python
"""Load the PiPER robot config in Isaac Sim and move every joint, before any RL.

This checks Stage A of the lab: the URDF converts, the robot stands on its
base, every joint follows its position target, and the link names that the
task config needs (base_link, link6) exist.

The script builds a tiny scene (ground, light, one PiPER) and then, joint by
joint, sends a slow sine-wave target across most of that joint's range. At the
end it prints how well each joint tracked its target.

Usage (from C:\\isaacsimlab\\isaac_so_arm101, with the GUI):
    & $py C:\\SO101wClaudeIsaac\\labs\\piper-lift\\tools\\test_piper_joints.py
Without the GUI (prints the table only):
    & $py C:\\SO101wClaudeIsaac\\labs\\piper-lift\\tools\\test_piper_joints.py --headless
"""

import argparse
import math

from isaaclab.app import AppLauncher

parser = argparse.ArgumentParser(description="Sweep each PiPER joint and report tracking.")
parser.add_argument("--seconds_per_joint", type=float, default=3.0, help="Sweep time for each joint.")
AppLauncher.add_app_launcher_args(parser)  # adds --headless, --device, ...
args_cli = parser.parse_args()

# Isaac Sim must start before anything else from Isaac Lab is imported
app_launcher = AppLauncher(args_cli)
simulation_app = app_launcher.app

import isaaclab.sim as sim_utils  # noqa: E402
from isaaclab.assets import Articulation  # noqa: E402

from isaac_so_arm101.robots.agilex_piper import PIPER_CFG  # noqa: E402


def main() -> None:
    # --- Simulation and scene -------------------------------------------------
    sim = sim_utils.SimulationContext(sim_utils.SimulationCfg(dt=0.01, device=args_cli.device))
    sim.set_camera_view(eye=[1.0, 1.0, 0.7], target=[0.15, 0.0, 0.2])
    # Ground 1 m below the base, so the arm can swing through its full range without hitting it
    ground = sim_utils.GroundPlaneCfg()
    ground.func("/World/GroundPlane", ground, translation=(0.0, 0.0, -1.0))
    light = sim_utils.DomeLightCfg(intensity=3000.0, color=(0.75, 0.75, 0.75))
    light.func("/World/Light", light)
    robot = Articulation(PIPER_CFG.replace(prim_path="/World/Robot"))

    sim.reset()  # converts the URDF, creates the physics objects
    dt = sim.get_physics_dt()

    # --- Names: these must match what the task config uses --------------------
    print("\n[test_piper_joints] joints:", robot.joint_names)
    print("[test_piper_joints] bodies:", robot.body_names)
    for needed in ("base_link", "link6"):
        status = "OK" if needed in robot.body_names else "MISSING - fix the task config"
        print(f"[test_piper_joints] body '{needed}': {status}")

    default_pos = robot.data.default_joint_pos.clone()
```

```
    limits = robot.data.soft_joint_pos_limits[0]  # (num_joints, 2): lower, upper

    # Let the arm settle in its start pose for 1 s
    robot.set_joint_position_target(default_pos)
    for _ in range(int(1.0 / dt)):
        robot.write_data_to_sim()
        sim.step()
        robot.update(dt)
    link6 = robot.body_names.index("link6")
    print(f"[test_piper_joints] link6 position at start pose (m): {robot.data.body_pos_w[0, link6].tolist()}")

    # --- Sweep each joint in turn -------------------------------------------
    results = []
    steps = int(args_cli.seconds_per_joint / dt)
    for j, name in enumerate(robot.joint_names):
        lo, hi = limits[j, 0].item(), limits[j, 1].item()
        mid, amp = (lo + hi) / 2.0, 0.35 * (hi - lo)  # use 70 % of the range
        max_err = 0.0
        for k in range(steps):
            if not simulation_app.is_running():
                return
            target = default_pos.clone()
            target[0, j] = mid + amp * math.sin(2.0 * math.pi * k / steps)
            robot.set_joint_position_target(target)
            robot.write_data_to_sim()
            sim.step()
            robot.update(dt)
            if k > steps // 10:  # ignore the first moment while the joint starts moving
                max_err = max(max_err, abs(robot.data.joint_pos[0, j].item() - target[0, j].item()))
        results.append((name, lo, hi, max_err))
        print(f"[test_piper_joints] swept {name}")

    # --- Report -------------------------------------------------------------
    print("\n  joint     lower     upper   max tracking error")
    for name, lo, hi, err in results:
        unit = "m" if name in ("joint7", "joint8") else "rad"
        flag = "" if err < 0.25 * (hi - lo) else "   <-- check stiffness / limits"
        print(f"  {name:<8}{lo:>8.3f}{hi:>10.3f}{err:>12.4f} {unit}{flag}")
    print("\n[test_piper_joints] done. Close the window (or it closes itself when headless).")

if __name__ == "__main__":
    main()
    simulation_app.close()
```

Stage B: The task

You reuse the generic lift task from topic 3 (`LiftEnvCfg` in `tasks\lift\lift_env_cfg.py`): table, observations, rewards, terminations and curriculum stay the same. You only write the robot-specific part.

Create the folder `$pkg\tasks\lift_piper\` with an `agents\` subfolder:

```
$pkg\tasks\lift_piper\
    __init__.py                 Step 8: registration
    joint_pos_env_cfg.py        Step 6: the PiPER lift task
    agents\__init__.py          Step 7
    agents\rsl_rl_ppo_cfg.py    Step 7: PPO settings
```

Step 6: The PiPER lift task (`PiperLiftCubeEnvCfg`)

joint_pos_env_cfg.py:

```
"""Lift-cube task for the AgileX PiPER, built on the generic lift task of isaac_so_arm101.

LiftEnvCfg (tasks/lift/lift_env_cfg.py) already defines the table scene,
observations, rewards, terminations and curriculum. Here we only fill in the
robot-specific parts, using the URDF names of the PiPER:
```

```
    robot          PIPER_CFG
    arm action     joint1..joint6, position targets
    gripper action joint7 / joint8, open or closed
    ee frame       base_link -> link6, plus 0.12 m to the point between the fingers
    cube           NVIDIA DexCube at 0.8 scale (4.8 cm), in front of the robot
    ranges         target and cube positions sized to the PiPER's reach
"""

import isaaclab_tasks.manager_based.manipulation.lift.mdp as mdp
from isaaclab.assets import RigidObjectCfg
from isaaclab.sensors.frame_transformer.frame_transformer_cfg import FrameTransformerCfg, OffsetCfg
from isaaclab.sim.schemas.schemas_cfg import RigidBodyPropertiesCfg
from isaaclab.sim.spawners.from_files.from_files_cfg import UsdFileCfg
from isaaclab.utils import configclass
from isaaclab.utils.assets import ISAAC_NUCLEUS_DIR


from isaac_so_arm101.robots.agilex_piper import PIPER_CFG
from isaac_so_arm101.tasks.lift.lift_env_cfg import LiftEnvCfg


from isaaclab.markers.config import FRAME_MARKER_CFG  # isort: skip


# Distance from the link6 origin to the point between the finger pads, along link6's z axis.
# The finger joints sit 0.1358 m out; the pads reach back about 0.07 m from there.
GRASP_POINT_OFFSET = (0.0, 0.0, 0.12)



@configclass
class PiperLiftCubeEnvCfg(LiftEnvCfg):
    def __post_init__(self):
        # post init of parent
        super().__post_init__()

        # --- Robot ---------------------------------------------------------
        self.scene.robot = PIPER_CFG.replace(prim_path="{ENV_REGEX_NS}/Robot")


        # --- Actions -------------------------------------------------------
        # Arm: target = start pose + 0.5 x action, for each of the 6 arm joints
        self.actions.arm_action = mdp.JointPositionActionCfg(
            asset_name="robot",
            joint_names=["joint[1-6]"],
            scale=0.5,
            use_default_offset=True,
        )
        # Gripper: the policy outputs open or close; both fingers move together
        self.actions.gripper_action = mdp.BinaryJointPositionActionCfg(
            asset_name="robot",
            joint_names=["joint7", "joint8"],
            open_command_expr={"joint7": 0.035, "joint8": -0.035},
            close_command_expr={"joint7": 0.0, "joint8": 0.0},
        )


        # --- Target command (where the cube must be carried) ---------------
        # Measured in the robot base frame. The PiPER faces +x and reaches about 0.6 m.
        self.commands.object_pose.body_name = ["link6"]
        self.commands.object_pose.ranges.pos_x = (0.25, 0.40)
        self.commands.object_pose.ranges.pos_y = (-0.15, 0.15)
        self.commands.object_pose.ranges.pos_z = (0.15, 0.30)


        # --- Cube ----------------------------------------------------------
        self.scene.object = RigidObjectCfg(
            prim_path="{ENV_REGEX_NS}/Object",
            init_state=RigidObjectCfg.InitialStateCfg(pos=[0.30, 0.0, 0.025], rot=[1, 0, 0, 0]),
            spawn=UsdFileCfg(
                usd_path=f"{ISAAC_NUCLEUS_DIR}/Props/Blocks/DexCube/dex_cube_instanceable.usd",
                scale=(0.8, 0.8, 0.8),  # 4.8 cm cube; the fingers open to 7 cm
                rigid_props=RigidBodyPropertiesCfg(
                    solver_position_iteration_count=16,
                    solver_velocity_iteration_count=1,
                    max_angular_velocity=1000.0,
                    max_linear_velocity=1000.0,
                    max_depenetration_velocity=5.0,
                    disable_gravity=False,
```

```
            ),
        ),
    )
    # Cube start position is randomised around (0.30, 0, 0.025) at every reset
    self.events.reset_object_position.params["pose_range"] = {
        "x": (-0.08, 0.08),
        "y": (-0.15, 0.15),
        "z": (0.0, 0.0),
    }

    # --- End-effector frame (used by the reaching reward) -------------------
    marker_cfg = FRAME_MARKER_CFG.copy()
    marker_cfg.markers["frame"].scale = (0.05, 0.05, 0.05)
    marker_cfg.prim_path = "/Visuals/FrameTransformer"
    self.scene.ee_frame = FrameTransformerCfg(
        prim_path="{ENV_REGEX_NS}/Robot/base_link",
        debug_vis=True,
        visualizer_cfg=marker_cfg,
        target_frames=[
            FrameTransformerCfg.FrameCfg(
                prim_path="{ENV_REGEX_NS}/Robot/link6",
                name="end_effector",
                offset=OffsetCfg(pos=GRASP_POINT_OFFSET),
            ),
        ],
    )

    # --- Rewards sized for a bigger arm and cube ----------------------------
    # Wider reaching kernel (the PiPER starts further from the cube than the SO101)
    self.rewards.reaching_object.params["std"] = 0.1
    # Cube centre rests at 0.024 m; count it as lifted above 0.04 m
    self.rewards.lifting_object.params["minimal_height"] = 0.04
    self.rewards.object_goal_tracking.params["minimal_height"] = 0.04
    self.rewards.object_goal_tracking_fine_grained.params["minimal_height"] = 0.04


@configclass
class PiperLiftCubeEnvCfg_PLAY(PiperLiftCubeEnvCfg):
    def __post_init__(self):
        # post init of parent
        super().__post_init__()
        # make a smaller scene for play
        self.scene.num_envs = 50
        self.scene.env_spacing = 2.5
        # disable randomization for play
        self.observations.policy.enable_corruption = False
```

What changed compared with the SO101 version (`tasks\lift\joint_pos_env_cfg.py`):

Setting	SO101	PiPER	Why
Arm joints	`shoulder_.*`, `elbow_flex`, `wrist_.*`	`joint[1-6]`	URDF names (Step 2)
Gripper action	`gripper`: open 0.5, close 0.0 (rad)	`joint7`/`joint8`: open ±0.035, close 0 (m)	Two prismatic fingers
End-effector frame	`gripper_link` + (0.01, 0, -0.09)	`link6` + (0, 0, 0.12)	Point between the finger pads
Cube	DexCube × 0.5 (3 cm) at x 0.2	DexCube × 0.8 (4.8 cm) at x 0.3	Bigger gripper and reach
Target range x / y / z (m)	-0.1–0.1 / -0.3 to -0.1 / 0.2–0.35	0.25–0.4 / ±0.15 / 0.15–0.3	The PiPER faces +x
Reaching kernel `std`	0.05	0.1	The gripper starts further from the cube
"Lifted" height	0.025 m	0.04 m	The bigger cube's centre

Setting	SO101	PiPER	Why
			rests at 0.024 m

Question B1. What would happen to training if `minimal_height` stayed at 0.025 m with the 4.8 cm cube?

Step 7: The training config

`agents\__init__.py`:

```
"""Training (agent) configs for the PiPER lift-cube task."""
```

`agents\rsl_rl_ppo_cfg.py`:

```
"""PPO training settings for the PiPER lift-cube task (RSL-RL)."""

from isaaclab.utils import configclass
from isaaclab_rl.rsl_rl import (
    RslRlOnPolicyRunnerCfg,
    RslRlPpoActorCriticCfg,
    RslRlPpoAlgorithmCfg,
)



@configclass
class PiperLiftCubePPORunnerCfg(RslRlOnPolicyRunnerCfg):
    num_steps_per_env = 24          # steps each arm collects per iteration
    max_iterations = 2000           # SO101 lift uses 1500; the PiPER has 2 more joints to learn
    save_interval = 50              # checkpoint every 50 iterations
    experiment_name = "piper_lift"  # results go to logs/rsl_rl/piper_lift/
    empirical_normalization = False
    policy = RslRlPpoActorCriticCfg(
        init_noise_std=1.0,
        actor_hidden_dims=[256, 128, 64],
        critic_hidden_dims=[256, 128, 64],
        activation="elu",
    )
    algorithm = RslRlPpoAlgorithmCfg(
        value_loss_coef=1.0,
        use_clipped_value_loss=True,
        clip_param=0.2,
        entropy_coef=0.006,
        num_learning_epochs=5,
        num_mini_batches=4,
        learning_rate=1.0e-4,
        schedule="adaptive",
        gamma=0.98,
        lam=0.95,
        desired_kl=0.01,
        max_grad_norm=1.0,
    )
```

Step 8: Register the task names

`__init__.py` (in `tasks\lift_piper\`):

```
"""Register the PiPER lift-cube tasks with Gymnasium.


isaac_so_arm101/tasks/__init__.py imports every sub-package of tasks/
automatically, so placing this folder at tasks/lift_piper/ is enough for the
train, play, zero_agent and random_agent scripts to find these task names.
"""

import gymnasium as gym

from . import agents

gym.register(
    id="Isaac-PiPER-Lift-Cube-v0",
```

```
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    kwargs={
        "env_cfg_entry_point": f"{__name__}.joint_pos_env_cfg:PiperLiftCubeEnvCfg",
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:PiperLiftCubePPORunnerCfg",
    },
    disable_env_checker=True,
)

gym.register(
    id="Isaac-PiPER-Lift-Cube-Play-v0",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    kwargs={
        "env_cfg_entry_point": f"{__name__}.joint_pos_env_cfg:PiperLiftCubeEnvCfg_PLAY",
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:PiperLiftCubePPORunnerCfg",
    },
    disable_env_checker=True,
)
```

You do not need to edit anything else. `isaac_so_arm101\tasks\__init__.py` imports every folder inside `tasks\`, so the new names are registered automatically. Check:

```
& $py "$lab\tools\list_lab_envs.py"
```

You should see `Isaac-PiPER-Lift-Cube-v0` and `Isaac-PiPER-Lift-Cube-Play-v0`. (The project's own `list_envs` only shows names containing `SO-ARM`, so it does not list them.)

Step 9: Preview the task in the GUI

Arms that hold still:

```
& $py -m isaac_so_arm101.scripts.zero_agent --task Isaac-PiPER-Lift-Cube-v0 --num_envs 4
```

Arms that move randomly:

```
& $py -m isaac_so_arm101.scripts.random_agent --task Isaac-PiPER-Lift-Cube-v0 --num_envs 4
```

You should see 4 copies of a PiPER on a table, gripper pointing down above a cube, a frame marker between the fingers and a target marker in the air. Close the window before training.

Checkpoint. Check three things before you train:

1. The frame marker sits between the finger pads, not inside the wrist or below the table.
2. The cube is in front of the robot and within reach.
3. With `random_agent`, the fingers open and close.

If the marker is in the wrong place, change `GRASP_POINT_OFFSET` in Step 6 and preview again.

Stage C: Train

Close all other Isaac Sim windows first.

Step 10: Smoke test

```
& $py -m isaac_so_arm101.scripts.rsl_rl.train --task Isaac-PiPER-Lift-Cube-v0 --headless --num_envs 64 --max_iterations 10
```

It succeeds when you see `Learning iteration 9/10` and the script exits without an error. Results go to `logs\rsl_rl\piper_lift\`.

Step 11: Full training

```
& $py -m isaac_so_arm101.scripts.rsl_rl.train --task Isaac-PiPER-Lift-Cube-v0 --headless --num_envs 1024
```

- 2000 iterations. Expect it to take longer than the SO101 lift.
- If you see `CUDA out of memory`, use `--num_envs 512`.
- Note the run folder name printed at the start (for example `2026-10-20_14-05-11`).

Step 12: Watch progress

In a second terminal (do the terminal set-up first):

```
& $py C:\SO101wClaudeIsaac\tools\tb_summary.py --logs logs\rsl_rl\piper_lift --filter Episode_Reward
```

Or TensorBoard (then open http://localhost:6006):

```
& $py -m tensorboard.main --logdir logs\rsl_rl\piper_lift
```

The rewards should rise in this order:

Metric	Rises when
`Episode_Reward/reaching_object`	first: the gripper learns to go to the cube
`Episode_Reward/lifting_object`	next: the cube is grasped and lifted
`Episode_Reward/object_goal_tracking`	last: the cube is carried to the target

Task C1. Record the iteration at which `lifting_object` first rises above about 10 % of its final value. Compare it with your SO101 run from topic 3.

If `lifting_object` is still near 0 at the end, continue training:

```
& $py -m isaac_so_arm101.scripts.rsl_rl.train --task Isaac-PiPER-Lift-Cube-v0 --headless --num_envs 1024 --resume --
load_run <run_folder> --checkpoint model_1999.pt --max_iterations 1000
```

Stage D: Play

Step 13: Watch the trained PiPER

```
& $py -m isaac_so_arm101.scripts.rsl_rl.play --task Isaac-PiPER-Lift-Cube-Play-v0 --num_envs 4
```

It loads the last checkpoint of the most recent run in `logs\rsl_rl\piper_lift\`. To choose one: `--load_run <run_folder> --checkpoint model_1000.pt`.

Step 14: Video and exported policy

```
& $py -m isaac_so_arm101.scripts.rsl_rl.play --task Isaac-PiPER-Lift-Cube-Play-v0 --num_envs 4 --headless --video --
video_length 500
```

The run folder then contains `exported\policy.pt`, `exported\policy.onnx` and `videos\play\*.mp4`.

Exercises

1. **Early vs late.** Play `model_500.pt` and the final checkpoint. Describe the behaviour of each in two sentences.
2. **Cube size.** Make a variant with the cube at scale 0.6. Which other settings must change with it? (Hint: Step 6, `minimal_height` and the cube's start height.) Register it as `Isaac-PiPER-Lift-Cube-Small-v0` and train it.
3. **Motor gains.** Halve the stiffness of `shoulder_elbow` and run Step 5 again. What happens to the tracking error, and what would you expect for training?

4. **Reward hacking.** Set `minimal_height` to 0.025 in a variant, train 500 iterations, and watch the video. Does the arm really lift the cube?

For variants, follow "Make your own variant" in topic 3: subclass `PiperLiftCubeEnvCfg`, change values in `__post_init__`, and register a new name.

What to hand in

Evidence	From
Completed table A1 and answers to A2, B1	Steps 2, 6
Screenshot of the `test_piper_joints.py` table with both bodies `OK`	Step 5
Screenshot of the preview with the frame marker between the fingers	Step 9
`tb_summary.py` output at the end of training, and the run folder name	Step 12
20–30 s video of the trained policy	Step 14
Answers to two exercises	Exercises

Troubleshooting

Problem	Cause and fix
`prepare_piper_urdf.py`: `HTTP Error 404` or timeout	No internet or GitHub blocked. Use `--source` with a local copy of `piper_description`
Robot invisible, or mesh errors at start-up	Mesh paths wrong. Run Step 2: every mesh must exist (no `FILE NOT FOUND`)
`body 'base_link': MISSING` in Step 5	`dummy_link` still in the URDF. Re-run Step 1
`ModuleNotFoundError: isaac_so_arm101.robots.agilex_piper`	`__init__.py` missing in `robots\agilex_piper\`, or files in the wrong folder
Task not found (`NameNotFound`)	`__init__.py` missing in `tasks\lift_piper\` or `agents\`; check with `list_lab_envs.py`
`ValueError` about joint names not found	A name in the config does not match the URDF (typo, or `joint[1-6]` written differently). Compare with Step 2
The arm sags or shakes in Step 5	Stiffness too low (sag) or damping too low (shake) for that joint group
Fingers never open fully	`soft_joint_pos_limit_factor` below 1.0, or open values with the wrong signs
Frame marker not between the fingers	Change `GRASP_POINT_OFFSET` in `joint_pos_env_cfg.py`
`lifting_object` stays near 0	Train longer; check the fingers close on the cube in the preview; check the offset
`CUDA out of memory`	Close other Isaac Sim windows and lower `--num_envs`
`Unexpected token`	The `&` before `$py` is missing

Appendix A: Software installation (Windows 11)

This is the set-up used on this machine: Windows 11, Python 3.11 virtual environment, Isaac Sim 5.1 from pip, Isaac Lab 2.3 from source, and `isaac_so_arm101` on top. Run all commands in **PowerShell**.

A1. Check the computer

Item	Minimum	This machine
OS	Windows 11 64-bit (or Ubuntu 22.04/24.04)	Windows 11
GPU	NVIDIA RTX with RT cores. NVIDIA's minimum is RTX 4080-class with 16 GB	RTX 5060 Laptop, 8 GB (works with `--num_envs` ≤ 1024)
RAM	32 GB	
Disk	50 GB free SSD	
NVIDIA driver	580.88 or newer on Windows	592.01

Check the driver: `nvidia-smi`. Update it from https://www.nvidia.com/Download/index.aspx if needed.

A2. Install Python 3.11 and Git

1. Python **3.11** (64-bit) from https://www.python.org/downloads/windows/. Tick **Add python.exe to PATH**. Isaac Sim 5.1 needs exactly 3.11.
2. Git from https://git-scm.com/download/win.
3. Allow long file paths (once, in an **administrator** PowerShell):

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" -Name "LongPathsEnabled" -Value 1 -PropertyType
DWORD -Force
git config --system core.longpaths true
```

A3. Create the Python environment

```
New-Item -ItemType Directory -Force C:\isaacsimlab
py -3.11 -m venv C:\isaacsimlab\env_isaaclab
C:\isaacsimlab\env_isaaclab\Scripts\activate
python -m pip install --upgrade pip
```

If PowerShell refuses to run `activate`, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

A4. Install Isaac Sim 5.1 and PyTorch

With the environment active:

```
pip install "isaacsim[all,extscache]==5.1.0" --extra-index-url https://pypi.nvidia.com
pip install -U torch==2.7.0 torchvision==0.22.0 --index-url https://download.pytorch.org/whl/cu128
```

First start (accept the licence; the first start can take 10 minutes or more):

```
$env:OMNI_KIT_ACCEPT_EULA = "YES"
isaacsim
```

Close the window when it has opened.

A5. Install Isaac Lab 2.3

```
cd C:\isaacsimlab
git clone https://github.com/isaac-sim/IsaacLab.git
cd IsaacLab
git checkout v2.3.0
.\isaaclab.bat --install
```

`--install` installs Isaac Lab and the RL libraries (including RSL-RL) into the active environment. Check it:

```
.\isaaclab.bat -p scripts\tutorials\00_sim\create_empty.py --headless
```

It should start, print simulation messages and exit without an error.

A6. Install `isaac_so_arm101`

```
cd C:\isaacsimlab
git clone https://github.com/MuammerBay/isaac_so_arm101.git
C:\isaacsimlab\env_isaaclab\Scripts\python.exe -m pip install -e C:\isaacsimlab\isaac_so_arm101 --no-deps
```

`--no-deps` stops pip from replacing the Isaac Lab you installed in A5 with another copy. Check:

```
cd C:\isaacsimlab\isaac_so_arm101
C:\isaacsimlab\env_isaaclab\Scripts\python.exe -m isaac_so_arm101.scripts.list_envs
```

The table should list `Isaac-SO-ARM101-Lift-Cube-v0` and the other SO101 tasks.

A7. Lab files

Copy the `labs\piper-lift\` folder to `C:\SO101wClaudeIsaac\labs\piper-lift\` (it is already there on the instructor PC). `tools\tb_summary.py` from `C:\SO101wClaudeIsaac\tools\` is used in Step 12.

A8. Final check

Check	Command	Expected
Driver	`nvidia-smi`	GPU listed, driver ≥ 580.88
Python	`& $py --version`	`Python 3.11.x`
PyTorch sees the GPU	`& $py -c "import torch; print(torch.cuda.is_available())"`	`True`
Isaac Lab	A5 tutorial	exits without error
SO101 tasks	A6 `list_envs`	SO101 tasks listed
SO101 training	topic 3, Step 9 smoke test	`Learning iteration 9/10`

You do not need to activate the environment for the lab: every command calls the environment's Python by its full path (`$py`).

Appendix B: Files created in this lab

```
C:\isaacsimlab\isaac_so_arm101\src\isaac_so_arm101\
    robots\agilex_piper\
        __init__.py
        piper.py                        robot config (PIPER_CFG)
        urdf\piper.urdf                 fixed URDF (Step 1)
        urdf\piper_description_original.urdf
        urdf\meshes\*.stl               10 meshes
    tasks\lift_piper\
        __init__.py                     registers Isaac-PiPER-Lift-Cube-v0 / -Play-v0
        joint_pos_env_cfg.py            PiperLiftCubeEnvCfg, PiperLiftCubeEnvCfg_PLAY
        agents\__init__.py
        agents\rsl_rl_ppo_cfg.py        PiperLiftCubePPORunnerCfg
C:\isaacsimlab\isaac_so_arm101\logs\rsl_rl\piper_lift\<run_folder>\
```

Appendix C: References

- AgileX PiPER description (URDF, meshes): https://github.com/agilexrobotics/piper_ros/tree/master/src/piper_description
- isaac_so_arm101 (SO101 tasks this lab builds on): https://github.com/MuammerBay/isaac_so_arm101
- Isaac Lab installation (pip): https://isaac-sim.github.io/IsaacLab/v2.3.2/source/setup/installation/pip_installation.html
- Isaac Lab tutorial, interacting with an articulation: https://isaac-sim.github.io/IsaacLab/v2.3.2/source/tutorials/01_assets/run_articulation.html
- Isaac Lab tutorial, manager-based RL environment: https://isaac-sim.github.io/IsaacLab/v2.3.2/source/tutorials/03_envs/create_manager_rl_env.html

- Isaac Sim 5.1 URDF importer: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/ext_isaacsim_asset_importer_urdf.html
- Isaac Sim 5.1 requirements: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/requirements.html